

Konzeption und Entwicklung eines interaktiven (Visualisierungs-)Systems für Exploded Views im Web.

Maximilian Elias Kellner

Modul: Praxisprojekt

Betreuer: Prof. Christian Noss

29. Oktober 2025

Zusammenfassung

In einer Zeit, in der die meisten Unternehmen und Produkte eine Onlinepräsenz haben, erwarten Nutzer grafisch ansprechende und interessante Benutzeroberflächen. Dabei stoßen statische Produktbilder oft an ihre Grenzen. Eine mögliche Alternative für interaktive und immersive Erfahrungen sind dreidimensionale Exploded-View-Animationen. Das Erstellen dieser ist jedoch aufwendig und nimmt viel Zeit in Anspruch. Um dieses Problem zu lösen, wurde in dieser Arbeit ein modulares System entwickelt, welches das Erstellen von Exploded Views mit Beschriftungselementen standardisiert ermöglicht und viele Konfigurationsmöglichkeiten für Animation und Interaktion bietet. Das Ergebnis ist ein wiederverwendbares npm-Paket, welches einfach in bestehende Systeme installiert und integriert werden kann.

Inhaltsverzeichnis

1	Einleitung	4
2	Theoretischer und praktischer Hintergrund	5
2.1	Begriffsklärung und Definition	5
2.2	Historische Entwicklung	5
2.3	Moderne Anwendungsfelder im Web	7
2.4	Technische Umsetzung und Herausforderungen im Web	7
2.4.1	Technologische Grundlagen	7
2.4.2	Praktische Anforderungen	8
3	Methodischer Rahmen	9
3.1	Vorgehensmodell: POCs und Iterative Prototypenentwicklung	9
4	Konzeption	10
4.1	Einfluss der Forschungsfragen auf die Konzeption	10
4.2	Entwicklungsprozess	10
4.3	Konzeptionelle Fragestellungen und Lösungsansätze	11
4.4	Verwendete Technologien	13
4.4.1	JavaScript (Primäre Programmiersprache)	13
4.4.2	Three.js (WebGL Framework)	13
4.4.3	GLB/GLTF (3D-Dateiformat)	14
4.4.4	Anime.js (Animationsbibliothek)	14
4.4.5	Tweakpane (Debug und Setup UI-Elemente)	14
4.4.6	Stats.js (UI-Elemente zur Performance Messung)	15
4.4.7	HTML/CSS (UI-Elemente)	15
4.4.8	JSON (Dateiformat für Daten und Einstellungen)	15
4.4.9	Blender (Beispiel für 3D-Software)	16
5	Durchführung	17
5.1	Proof of concepts (POCs)	17
5.1.1	Rendern einer 3D-Datei	17
5.1.2	Animation von 3D-Modellen über Namenskonvention	18
5.1.3	2D-Beschriftungselemente	21
5.1.4	3D-Beschriftungselemente	22
5.1.5	2D-Beschriftungselemente mit Hervorhebung	24
5.1.6	Interaktives Menü	26
5.1.7	Animationsbibliothek	28
5.2	Prototyp	28
5.2.1	3D-Darstellung	29
5.2.2	Beschriftungselemente	30
5.2.3	Interaktionen	32
5.2.4	Animation	32
5.2.5	Modularisierung	35
5.2.6	Dateneinbindung und Nutzung	35
5.2.7	Zusammenfassung und Bewertung	35

6	Ergebnisse und Diskussion	36
6.1	Bewertung	36
6.2	Limitationen	36
7	Fazit und Ausblick	36
7.1	Zusammenfassende Beantwortung der Forschungsfragen	36
7.2	Ausblick	37

1 Einleitung

Kontext

In einer zunehmend digitalisierten Welt spielen die Onlinepräsenz und das Digitalisieren von Unternehmen, Institutionen und Inhalten eine zentrale Rolle. Entsprechend fließen viele Ressourcen in das visuelle und funktionale Design von Webseiten, welche für viele Anwendungsfälle unverzichtbar sind. Während in den vergangenen Jahren ein minimalistischer, flacher Gestaltungsansatz dominierte, wie etwa aus den Material Design 2 Guidelines hervorgeht (Google, 2014), gewinnen derzeit interaktive, dreidimensionale und physikalisch korrekte Komponenten, wie sie beispielsweise in Apples “Liquid Glass” vorzufinden sind (Apple, 2025), an Bedeutung.

Besonders getrieben vom Aufschwung autonomer VR-Headsets (PwC, 2025) und der stetig steigenden Leistung mobiler Endgeräte (Halpern et al., 2016) hält dieser Trend Einzug in diverse Bereiche. Ob mobile Betriebssysteme, Produktdarstellungen, E-Learning, Online-Konfiguratoren oder (Reparatur-) Anleitungen. Es gibt viele Anwendungsbereiche, welche mit interaktiven 3D-Elementen erfahrbarer, emotionaler und verständlicher gestaltet werden können. Ein Ansatz bei der Darstellung von 3D-Modellen auf Bildschirm und Papier (vgl. Abb. 3) sind Exploded Views. Diese werden in der Planung und Entwicklung von industriellen Teilen (vgl. Abb. 2) und Produkten (vgl. Abb. 4) verwendet und haben auch Einzug in CAD-Programme und Webdesign gefunden.

Problemstellung

Laut (McKeon, 2018) legen über 80% der Nutzer großen Wert auf Produktdarstellungen und Beschreibungen auf Webseiten mit ansprechendem und aktuellem Design. Trotz des hohen Potenzials ist das Modellieren, Texturieren, Animieren und Integrieren von 3D-Modellen mit einer Erhöhung der Komplexität und des Zeitaufwands verbunden. Besonders Animationen und Interaktionen können die Komplexität stark erhöhen und erfordern Fachkenntnisse, Zeit und weitere Ressourcen. Bislang scheint es noch keine standardisierte, einfache Lösung zur Integration von interaktiven Exploded Views zu geben.

Das Ziel dieser Arbeit ist es daher, die Konzeption und Entwicklung eines interaktiven (Visualisierungs-)Systems für Exploded Views im Web, das ansprechende Animationen, flüssige und organische Benutzerinteraktionen sowie eine einfache und übersichtliche Integration für Entwickler ermöglicht. Ergänzend werden verschiedene Beschriftungselemente entwickelt und verglichen, welche die Integration von Informationen und Daten ermöglichen. Dabei werden insbesondere die folgenden Fragestellungen untersucht.

- Wie lassen sich Exploded Views im Web modular und wiederverwendbar umsetzen und welche Best Practices sowie Herausforderungen existieren dabei?
- Welche Vorteile und Möglichkeiten bieten 3D Modelle und Exploded Views im Hinblick auf Storytelling?
- Wie lassen sich Interaktivität und Animation (z.B. Scroll- oder Hover-Effekte) integrieren?

2 Theoretischer und praktischer Hintergrund

In diesem Kapitel werden die theoretischen und praktischen Grundlagen von Exploded Views und die historische Entwicklung mit einigen Beispielen aus verschiedenen Epochen und Domänen beschrieben.

2.1 Begriffsklärung und Definition

Exploded Views beschreiben eine Darstellungsform für dreidimensionale Objekte, bei der ein Objekt in seine relevanten Komponenten zerlegt wird. Diese Komponenten werden dann auf vordefinierten Achsen aufgereiht. Dadurch wird es möglich, die räumliche Anordnung und das funktionale Zusammenspiel der Komponenten zu visualisieren und somit den Aufbau, die Beziehungen und die Funktionen der Komponenten innerhalb eines komplexen Objekts nachvollziehbar darzustellen. So kann im zweidimensionalen Raum ein dreidimensionales Objekt verständlich zerlegt werden.

2.2 Historische Entwicklung

Bereits in der Renaissance wurden erste Formen von Exploded Views verwendet, um Erfindungen darzustellen und zu dokumentieren. Ingenieure und Erfinder nutzten Zeichnungen, um komplexe Mechanismen zu veranschaulichen. Da diese statischen Zeichnungen jedoch nur schlecht die verschiedenen Ebenen von mechanischen Bauteilen darstellen konnten, verwendete unter anderem Leonardo da Vinci oftmals Skizzen (vgl. Abb. 1), welche die Einzelteile der Mechanismen nebeneinander aufgereiht zeigten. Diese Zeichnungen können als Vorläufer der heutigen Exploded Views betrachtet werden.

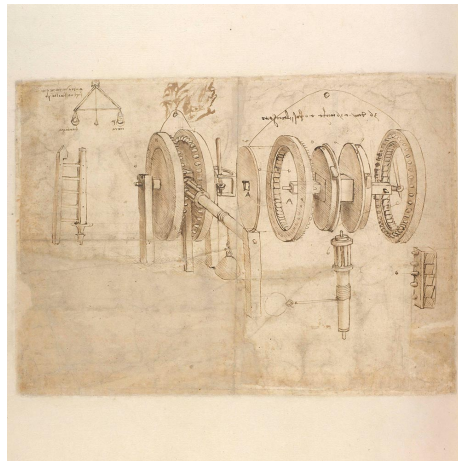


Abbildung 1: Technische Zeichnung von Leonardo da Vinci aus dem Codex Atlanticus (ca. 1478–1519). Die Darstellung ist ein früher Vorläufer der modernen Explosionsdarstellung (Quelle der Abbildung: (The Visual Agency and The Venneranda Biblioteca Ambrosiana, n. d.)).

Eine Möglichkeit, komplexe Objekte geordnet mit allen Einzelteilen auf Papier darzustellen, um deren Aufbau und Funktion zu dokumentieren, war besonders

mit der Industrialisierung und der darauffolgenden Globalisierung und Massenproduktion gefragt. In diesem Kontext etablierten sich Exploded Views als wichtiger Bestandteil von Patentanmeldungen (vgl. Abb. 2). Diese Darstellungsmethode ermöglichte es, Objekte präzise und nachvollziehbar festzuhalten, da nicht nur die Erfindung als Ganzes, sondern auch die spezifische Anordnung der Komponenten festgehalten werden konnte. Exploded Views tragen seitdem zum Schutz des geistigen Eigentums bei.

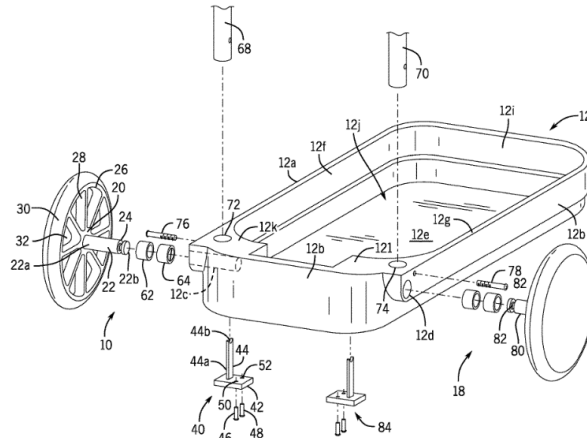


Abbildung 2: Explosionsdarstellung einer Rad Konstruktion auf Seite 1 des US Patents US 2025/0262893 A1 (Quelle der Abbildung: (Rolon et al., n. d.)).

Heutzutage sind Exploded Views auch im Alltag zu finden. Anleitungen für Spielzeug zum Aufbau oder zur Reparatur beinhalten diese oft, um dem Nutzer das Zusammenspiel von Komponenten zu verdeutlichen. Sie ermöglichen es, das dreidimensionale Objekt in einer PDF oder einem Buch darzustellen (vgl. Abb. 3).

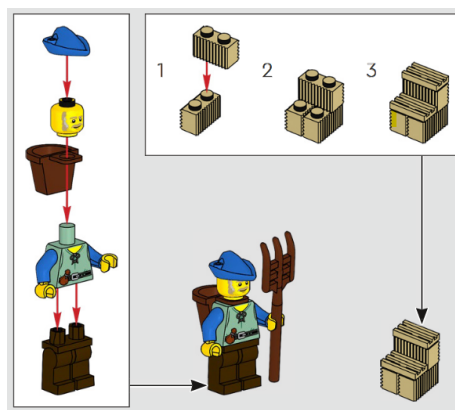


Abbildung 3: Explosionsdarstellung eines Bauschritts auf Seite 10 der Bauanleitung für das LEGO Set 10305 Burg der Löwenritter (Quelle der Abbildung: (The LEGO Group, 2022)).

2.3 Moderne Anwendungsfelder im Web

Ein aktuelles Anwendungsfeld von Exploded Views stellt das Webdesign dar. Diese Implementation eröffnet eine zusätzliche Ebene, die Interaktion. Interaktive Exploded Views erweitern die Anwendungsmöglichkeiten und Funktionen immens. Nutzer können das Objekt je nach Implementierung nun vom Ausgangszustand bis zum vollständig explodierten Zustand in verschiedenen Stufen betrachten. Zudem besteht die Möglichkeit, das Objekt im dreidimensionalen Raum zu drehen und es durch Zoomfunktionen und andere Manipulationen verständlicher und greifbarer darzustellen. Diese Implementation wird besonders auf Produktwebseiten verwendet, um Nutzern den Aufbau des Produktes und dessen Eigenschaften zu präsentieren (vgl. Abb. 4).

Diese interaktiven Funktionen tragen dazu bei, dass Nutzer länger auf der gegebenen Webseite verweilen und sich intensiver mit den Inhalten auseinandersetzen. Studien zeigen, dass interaktive Elemente allgemein die Nutzerzufriedenheit und die Einstellung zur Webseite positiv beeinflussen (vgl. (Teo et al., 2003)). Zudem werden Nutzer kognitiv gefordert und das Kaufverhalten kann positiv beeinflusst werden (vgl. (Jiang et al., 2010)).



Abbildung 4: Explosionsdarstellung des Kameramoduls des OnePlus 7 Pro (Quelle der Abbildung: (OnePlus, 2019)).

2.4 Technische Umsetzung und Herausforderungen im Web

Die Realisierung von interaktiven 3D-Anwendungen stellt Entwickler, besonders im Web, vor einige Herausforderungen. Es gibt jedoch diverse Technologien, welche bei der Entwicklung helfen und komplexe Aufgaben erleichtern. Die Vor- und Nachteile der ausgewählten Technologien sowie weitere wichtige Konzeptionelle Fragen werden im Kapitel 4 behandelt.

2.4.1 Technologische Grundlagen

Die Grundlage für die Darstellung von 3D-Grafiken im Web bildet die Programmierschnittstelle WebGL (Web Graphics Library). WebGL ermöglicht die Nut-

zung der Grafikkarte des Nutzers und erlaubt so eine performante Darstellung komplexer 3D-Szenen ohne weitere Browserplugins. Da die direkte Programmierung in WebGL aufwendig ist, haben sich JavaScript-Bibliotheken, welche darauf aufbauen, etabliert. Dabei haben sich Three.js (vgl. 4.4.2 und Babylon.js zum Standard entwickelt. Diese Frameworks stellen Werkzeuge für die Erstellung von 3D-Szenen mit interaktiven Elementen, die Verwaltung von Kameras, Lichtquellen und das Laden von 3D-Modellen zur Verfügung.

2.4.2 Praktische Anforderungen

Um eine positive Nutzererfahrung zu gewährleisten, müssen bei der Konzeption und Entwicklung dieser Systeme diverse Anforderungen berücksichtigt werden.

- **Ladezeiten:** 3D-Modelle können, insbesondere durch einen hohen Detailgrad und hochauflösende Texturen, große Datenmengen beanspruchen. Die Optimierung großer Modelle ist daher entscheidend, um die Ladezeiten der Webseite gering zu halten und keine Nutzer aufgrund langer Ladezeiten zu verlieren. Ein Kompromiss kann hier auch eine Ladeanimation sein, welche für das Element eingebaut wird. Informationen über den Systemstatus scheinen Nutzern vorübergehend auszureichen, so Nielsen (1994b).
- **Browserkompatibilität:** Obwohl WebGL von nahezu allen modernen Browsern unterstützt wird (vgl. (Can I Use, 2025)), ist die Leistung abhängig vom Endgerät und kann variieren. Es sollten also Tests durchgeführt werden, um eine gute Nutzererfahrung auf der Zielplattform zu gewährleisten.
- **Benutzbarkeit (usability):** Die Steuerung der 3D-Szene sollte einfach und intuitiv gestaltet sein. Eine unklare oder komplizierte Interaktion kann zu Frustration führen und den positiven Effekt der 3D-Komponente vernichten. Animationen, welche den Nutzer vom Scrollen oder anderweitigen Interagieren abhalten, können ebenfalls die Nutzererfahrung verschlechtern und sollten vermieden werden. Eine Umfrage aus dem Jahr 2018 mit 612 Teilnehmern hat ergeben, dass 94% der Nutzer einfache und intuitive Navigation als wichtig ansehen. (vgl. (McKeon, 2018))
- **Modularität:** Eine modulare Softwarearchitektur ist wichtig, um die Wiederverwendbarkeit, Wartbarkeit und Erweiterungen zu ermöglichen.

3 Methodischer Rahmen

In diesem Kapitel wird das methodische Vorgehen für die Konzeption und Entwicklung des Systems dargelegt. Es wird das Vorgehensmodell für die Umsetzung beschrieben.

3.1 Vorgehensmodell: POCs und Iterative Prototypenentwicklung

Da besonders zu Beginn des Projektes die Anforderungen und Lösungsansätze noch wenig präzise waren, wurde zur Domäne recherchiert, um einen Überblick über relevante Themen und mögliche Lösungsansätze zu erhalten.

Darauf basierend wurden schrittweise Proof of Concepts (POCs) für diverse Komponenten wie das Rendern der 3D-Modelle, der Beschriftungselemente, UI-Komponenten usw. entwickelt. Dadurch konnten Lösungsansätze und Technologien schnell im Rahmen des Projekts getestet und evaluiert werden.

Diese Erkenntnisse wurden dann kombiniert, um einen Prototypen mit den relevanten Grundfunktionen zu erstellen. Dabei wurde die Struktur und Qualität der Codebasis verbessert, damit der Prototyp als Grundlage für weitere, komplexere POCs verwendet werden konnte. Die Ergebnisse der POCs konnten dann iterativ in den Prototypen ergänzt werden, um sich dem Projektziel anzunähern. Nach der POC-Phase wurde dieser Prototyp um weitere Funktionen und Verbesserungen ergänzt, um das Projektziel zu erreichen.

4 Konzeption

Die für die Entwicklung des Systems benötigten Technologien werden beschrieben, und deren Auswahl sowie Designentscheidungen werden begründet. Diese Technologien haben sich bei der Recherche und in den POCs bewährt und wurden für die Entwicklung des Prototypen verwendet.

4.1 Einfluss der Forschungsfragen auf die Konzeption

Die in der Einleitung formulierte Problemstellung und die daraus resultierenden Ziele und Forschungsfragen bilden die Grundlage der Konzeption und ziehen Anforderungen und Designentscheidungen nach sich.

- **F1: Wie lassen sich Exploded Views im Web modular und wiederverwendbar umsetzen und welche Best Practices sowie Herausforderungen existieren dabei?**

Zur Beantwortung der Forschungsfrage sollten bei der Auswahl und Entwicklung von Technologien und Lösungsansätzen die Performance, Usability und User Experience besonders beachtet werden. Zudem spielt die Einhaltung von Kriterien wie Codequalität und Wartbarkeit eine wichtige Rolle.

- **F2: Welche Vorteile und Möglichkeiten bieten 3D Modelle und Exploded Views im Hinblick auf Storytelling?**

Um die Vorteile von Exploded Views und interaktiven Elementen auf Webseiten zu bewerten, sollten Studien untersucht werden. Es sollte zudem überprüft werden, welche positiven Effekte interaktive Elemente und Animationen auf die Nutzererfahrung und das Verständnis des Inhalts haben.

- **F3: Wie lassen sich Interaktivität und Animation (z.B. Scroll- oder Hover-Effekte) integrieren?**

Zur Beantwortung dieser Forschungsfrage sollten POCs entwickelt werden, welche verschiedene Interaktionen und Animationen erproben.

4.2 Entwicklungsprozess

1. **Anforderungsanalyse und Konzeption:** Auf Basis der formulierten Problemstellung, der Ziele und der Forschungsfragen werden Anforderungen abgeleitet. Diese werden dann im Entwicklungsprozess beachtet und umgesetzt.
2. **Proof of concepts (POCs):** Die Kernfunktionen wie das Rendern von 3D-Modellen, Animationen und Beschriftungselementen werden in eigenständigen POCs erprobt, um die Machbarkeit und mögliche Lösungsansätze festzustellen.
3. **Prototyping:** Erfolgreiche POCs werden in ein System integriert und ermöglichen so den Aufbau eines Prototypen, welcher iterativ weiterentwickelt werden kann.
4. **Evaluation:** Die Ergebnisse werden bewertet und Verbesserungen werden vorgenommen.

4.3 Konzeptionelle Fragestellungen und Lösungsansätze

Im Verlauf der Konzeption und der POCs traten verschiedene Fragestellungen auf. Diese beeinflussten besonders die Designentscheidungen hinsichtlich der Interaktion, Darstellung und Strukturierung der 3D-Modelle sowie die Umsetzung der Beschriftungselemente.

3D-Modellstruktur Um eine Exploded View Animation zu erstellen, müssen Daten über Bewegungsrichtung, Distanz und Geschwindigkeit für viele Einzelteile gespeichert werden. Dafür muss ein System entwickelt werden, welche diese Daten übersichtlich und logisch bündelt, um Nutzern das erstellen und Bearbeiten von Animationen zu gewährleisten. Dabei könnte auf Eigenschaften wie den Namen des 3D-Objekts zugegriffen werden, alternativ könnten die Daten in einer externen Datei gespeichert werden, um sie vom Modell zu entkoppeln. Verschachtelungen und Eltern-Kind-Beziehungen müssen ebenfalls berücksichtigt werden, da sie abhängig von der Implementierung Gruppierungen ermöglichen oder unerwartetes Verhalten hervorrufen (vgl. Abb.19, Abb.20).

Interaktionskonzepte (Scroll, Click, Trigger, Slider) Es bestehen verschiedene Möglichkeiten, mit interaktiven Komponenten und Animationen auf Webseiten zu interagieren. Besonders im Hinblick auf die Usability und im 3D-Kontext gibt es jedoch einige Regeln zu beachten. Beispielsweise sollten Nutzer stets die Kontrolle behalten und Animationen oder Interaktionen kontrollieren oder umgehen können. Zudem sollten Interaktionen nach bekannten Mustern gekennzeichnet sein, so Nielsen (1994a). Basierend auf diesen Erkenntnissen entstehen gewisse Vor- und Nachteile für verschiedene Interaktionen.

Scroll: Vorteilhaft, da Nutzer mit einer bekannten Interaktionsform oft auch unabsichtlich die Interaktion entdecken und erkunden können. Nutzer können die Animation präzise kontrollieren, sie vorwärts und rückwärts abspielen oder pausieren. Nachteilig ist die Abhängigkeit von den Scrolleinstellungen und der Hardware des Nutzers. Damit die Animation flüssig abläuft, werden Mausemphasen mit Freilauf oder ein Touchpad benötigt. Dieses Problem kann jedoch mit Easing Effekten gemindert werden. Zoomfunktionen werden weniger intuitiv umsetzbar und es muss auf GUI Komponenten ausgewichen werden. Zudem ist es nicht möglich, die Komponente schnell zu überspringen, wenn die Animation an das Scrollen gekoppelt ist. Aufgrund dessen sollte diese Option nur in Popups oder mit anderweitigen Optionen zum Schließen oder Überspringen genutzt werden.

Click: Ermöglicht eine gezielte, bewusste Interaktion. Der Nutzer erhält einen klaren Call to Action (CTA) über Buttons, Overlays oder andere GUI Komponenten und kann die Animation dann abspielen. Diese Art der Animation gewährleistet einen flüssigen Ablauf von Start bis Ende. Die Dauer und das Easing der Animation können vom Entwickler festgelegt werden und sind bei jedem Ablauf gleich. Nachteile sind die fehlende Kontrolle des Nutzers, welche jedoch durch Pause- und Reverse-Funktionen verbessert werden kann und die weniger organische Integration, da der Nutzer die Animation aktiv starten muss.

Trigger: Trigger-Aktionen (z.B. durch Hovern oder Eintreten in den Viewport) können organische Möglichkeiten zum Starten einer Animation sein. Diese Form

der Interaktion birgt jedoch einige Nachteile. Der Nutzer hat wenig Kontrolle über den Ablauf der Animation. Diese Funktionen müssten über weitere GUI-Komponenten hinzugefügt werden. Zudem können diese Trigger versehentlich aktiviert oder zurückgesetzt werden, wodurch Interaktionen mit dem animierten Objekt in dem gewünschten Zustand schwieriger werden.

Slider: Slider bieten präzise Kontrolle über den Fortschritt der Animation. Sie sind eine gute Möglichkeit, um einige UX-Probleme von Scrollanimationen zu umgehen und können auch bei anderen Interaktionen als Ergänzung implementiert werden. Probleme wie der flüssige Ablauf der Animation bleiben jedoch bestehen.

Labeling und Textdarstellung (3D vs. 2D) Die Darstellung von Beschriftungen kann in zwei Kategorien eingeteilt werden, 2D- oder 3D-Beschriftungselemente. 3D-Beschriftungselemente werden vollständig in den dreidimensionalen Raum integriert und tragen zu einer außergewöhnlichen Nutzererfahrung bei. Sie sind in Relation zum 3D-Modell positioniert und folgen den Manipulationen wie Rotation, Zoom und Animation. Häufige Probleme bei dieser Methode sind die Überlagerung und Kollision von Beschriftungen und 3D-Modellen. Zudem kann die Lesbarkeit durch die Perspektive, Zoomstufe oder die Beleuchtung negativ beeinflusst werden. Zur Optimierung der Leistung und Lesbarkeit werden die Beschriftungen oft nur als Textur auf einer Fläche und nicht als komplexe 3D-Objekte dargestellt. Dadurch fehlen jedoch viele Textverarbeitungsfunktionen und die Skalierung wird erschwert. Eine Alternative bieten 2D-Beschriftungselemente. Diese Elemente überlagern die Szene und können somit unabhängig vom 3D-Modell und allen Effekten wie der Beleuchtung, Zoomstufe oder der Perspektive agieren. Sie bieten ebenfalls eine bessere Performance, da sie im HTML-DOM positioniert und manipuliert werden können. Auch die Textverarbeitungs- und Designfunktionen sind deutlich einfacher und performanter umsetzbar. Dadurch ist es möglich, ein Design zu schaffen, welches responsiv und mit mobilen Geräten kompatibel ist. Zudem können Screenreader besser auf statische Inhalte im HTML Code zugreifen, weshalb die Barrierefreiheit maßgeblich verbessert werden kann. Jedoch hat auch diese Implementation einige Nachteile. Die Zuordnung zwischen 3D-Objekt und Beschriftungen wird schwieriger umsetzbar. Es müssen Verbindungen oder eine Hervorhebung der ausgewählten Komponente implementiert werden, um die Zugehörigkeit zu verdeutlichen.

Beleuchtung und Kamera Bei der Beleuchtung können Kompatibilitätsprobleme zwischen der 3D-Software und Three.js auftreten. Es gibt verschiedene Beleuchtungsarten und die Lichtberechnung hängt stark vom verwendeten Shader und der Render-Engine ab. Dadurch kann es beispielsweise bei Szenen, welche in Blender mit der Eevee-Render-Engine gut belichtet sind, zu Über- oder Unterbelichtung nach dem Export in Three.js kommen. Lichter sollten in der Three.js Umgebung konfiguriert werden, um konstante Ergebnisse zu erhalten. Auch die Kamera sollte in der Three.js Szene konfiguriert werden, um die richtige Ausrichtung, Skalierung und Animation zu ermöglichen.

Konfiguration Der 3D-Kontext bietet viel Raum für Konfiguration. Es sollte eine Vorauswahl an Optionen getroffen werden, um eine einfache Integration zu gewährleisten. Diese Optionen sollten einfach konfigurierbar sein. Zudem sollte es die Möglichkeit geben, die Optionen für andere Szenen oder Tests zu kopieren. Die Optionen sollten an einem Zentralen Ort verwaltet werden können und ein klares Format haben.

4.4 Verwendete Technologien

Im folgenden werden die ausgewählten Technologien aufgezählt. es wird jeweils mit einer Auflistung von Vor- und Nachteilen die Entscheidung für diese Technologie begründet.

4.4.1 JavaScript (Primäre Programmiersprache)

Vorteile:

- **Plattformunabhängigkeit:** JavaScript ist der Industriestandard für Webanwendungen hinsichtlich der DOM-Manipulation und der Funktionalitäten, welche über HTML und CSS hinausgehen. Browser haben JavaScript standardmäßig aktiviert und Nutzer müssen keine Erweiterungen installieren.
- **Großes Ökosystem:** Es gibt ein großes Ökosystem an Bibliotheken, Frameworks, Hilfestellungen, Dokumentationen und anderen Tools, welche die Entwicklung erleichtern und verbessern können.
- **Modulstruktur:** Es ist möglich, den Code modular und wiederverwendbar zu gestalten.
- **Asynchrone Operationen:** JavaScript asynchrone Operationen, welche unter anderem für das Laden von Dateien vorteilhaft sind, da so die Benutzeroberfläche nicht blockiert werden muss.

Nachteile:

- **Dynamische Typisierung:** JavaScript hat keine festen Datentypen für Variablen, wodurch Inkonsistenzen und Laufzeitfehler entstehen können.

4.4.2 Three.js (WebGL Framework)

Vorteile:

- **Geringere Komplexität:** Der Umgang mit 3D-Modellen wird einfacher und leistungsfähiger.
- **Plattformunabhängigkeit und Kompatibilität:** Three.js bzw. WebGL ist laut Can I Use (2025) mit allen aktuellen und verbreiteten Browsern kompatibel. Zudem bietet die Entwicklung eines webbasierten Services Plattformunabhängigkeit und kann auf diversen Endgeräten genutzt werden.
- **Umfangreiche Funktionalität:** Three.js bietet diverse Funktionen wie Lichtquellen, Schatten, Materialien, Transparenzeffekte und Shader. Viele dieser Funktionen werden für das Projekt benötigt.

- **Umfangreiche Unterstützung:** Three.js hat eine große Community, welche Tutorials und Beispiele liefert. Diese Dokumentationen und Hilfestellungen beschleunigen den Entwicklungsprozess.

Nachteile:

- **Abhängigkeit von fremdem Code:** Neue Updates können potenziell Sicherheitslücken oder breaking changes beinhalten.

4.4.3 GLB/GLTF (3D-Dateiformat)

Vorteile:

- **Standardisierter Austausch:** GLB Dateien werden von den meisten 3D-Tools wie Blender und Three.js unterstützt.
- **Großer Funktionsumfang:** Die GLTF Komponente beinhaltet die 3D-Daten und wird in Kombination mit Texturen, Animationen und anderen wichtigen Inhalten in eine GLB Datei gekapselt. Dadurch ist es beispielsweise möglich, fertige 3D-Modelle aus Blender in das Projekt zu laden.
- **Performance:** Da GLB die gesamten Daten schließlich im Binärformat speichert, ist die Ladegeschwindigkeit vergleichsweise gut.

4.4.4 Anime.js (Animationsbibliothek)

Vorteile:

- **Größe:** Anime.js wird mit einer Größe von 27,13 KB angegeben (vgl. Garnier (2025)) und ist somit sehr klein und leistungsfähig im Vergleich zu anderen Alternativen. Dies ist wichtig, um die Ladezeiten der Webseite möglichst gering zu halten.
- **Einfache Syntax:** Es ist möglich, mit einem kompakten Codeabschnitt Animationen zu erstellen.

Nachteile:

- **Steile Lernkurve:** Mit der Dokumentation ist es möglich, kleine Animationen schnell zu entwickeln. Der volle Funktionsumfang ist jedoch deutlich schwieriger zu verstehen und umzusetzen.
- **Abhängigkeit von fremdem Code:** Neue Updates können potenziell Sicherheitslücken oder breaking changes beinhalten.
- **Keine 3D-Funktionen:** Anime.js bietet keine direkte Integration für 3D-Elemente.

4.4.5 Tweakpane (Debug und Setup UI-Elemente)

Vorteile:

- **schnelle Anpassung:** Tweakpane bietet Elemente, um Farben, Koordinaten, Zahlenwerte usw. schnell in einer Benutzeroberfläche anzupassen, damit während der Entwicklung und Integration verschiedene Werte erprobt werden können.

- **Einfache Integration:** Tweakpane bietet eine sehr flache Lernkurve und eine ausführliche Dokumentation. Dadurch ist es sehr gut integrierbar und anwendbar.
- **Modernes Interface:** Die gegebenen Komponenten haben bereits ein modernes und funktionales Design, welches durch eigenes CSS und Plugins erweitert werden kann.
- **Ordnerstruktur:** Das Menü kann in Ordner aufgeteilt werden. Diese Funktion wird zum Gruppieren und Ausblenden von Optionen benötigt.
- **Begrenzter Fremdcode:** Tweakpane verzichtet auf weitere Abhängigkeiten und wird aktiv weiterentwickelt.

Nachteile:

- **Abhängigkeit von fremdem Code:** Neue Updates können potenziell Sicherheitslücken oder breaking changes beinhalten.

4.4.6 Stats.js (UI-Elemente zur Performance Messung)

Vorteile:

- **Einfaches Setup:** Stats.js kann einfach eingerichtet und integriert werden.
- **Einfache Übersicht:** Es ist einfach, die Performance zu überwachen. Stats.js bietet beispielsweise Visuelles Feedback über die frames per second (FPS) und die Zeit, welche für das Rendern eines Bildes benötigt wurde.

Nachteile:

- **Abhängigkeit von fremdem Code:** Neue Updates können potenziell Sicherheitslücken oder breaking changes beinhalten.

4.4.7 HTML/CSS (UI-Elemente)

Vorteile:

- **Standardtechnologie:** Eine Kombination von HTML und CSS ist der Standard für das Design von Webanwendungen.
- **Kompatibilität:** HTML und CSS sind mit allen aktuellen Browsern kompatibel. Anime.js Bietet ebenfalls eine sehr gute Anbindung, um Komponenten zu animieren.

4.4.8 JSON (Dateiformat für Daten und Einstellungen)

Vorteile:

- **Plattformunabhängigkeit und Kompatibilität:** JSON ist im Web-Kontext in vielen Situationen der Standard und kann plattformunabhängig im Browser verwendet werden.

- **Einfache Datenverwaltung:** JSON bietet die Möglichkeit, Daten und Einstellungen geordnet zu verwalten. Die Daten können menschenlesbar benannt und angeordnet werden, können aber auch in diversen Programmiersprachen sehr gut ausgelesen und verwendet werden.

Nachteile:

- **Keine Validierung:** Die Daten werden weder validiert noch typisiert.
- **Große Dateien:** Große Dateien und Datenmengen können unübersichtlich werden.

4.4.9 Blender (Beispiel für 3D-Software)

Vorteile:

- **Kostenlos:** Blender ist im Gegensatz zu vielen Alternativen kostenlos nutzbar.
- **Open Source:** Blender wird in Zusammenarbeit mit einer großen Community entwickelt.
- **Umfangreiche Funktionalität:** Blender hat einen sehr großen Funktionsumfang. Dazu zählen Modellierung, Texturierung, Animation, Rendering und Videoschnitt. Zudem kann der Funktionsumfang durch diverse Erweiterungen ausgeweitet werden.
- **Kompatibilität:** Blender unterstützt viele Formate für den Import und Export, unter anderem GLB/GLTF, welche für dieses Webbasierte Projekt wichtig sind.

Nachteile:

- **Leistung:** Besonders für komplexe Projekte ist gute Hardware erforderlich.
- **Steile Lernkurve:** Aufgrund des hohen Funktionsumfangs ist es schwierig, eine Übersicht zu bekommen und die verschiedenen Funktionen zu lernen.

5 Durchführung

5.1 Proof of concepts (POCs)

Im folgenden Abschnitt werden die angefertigten POCs beschrieben. Es wird jeweils ein kurzer Überblick über die Zielsetzung, die Durchführung und die erhaltenen Ergebnisse gegeben. Zudem werden diese Ergebnisse bewertet, um den Erfolg oder Misserfolg sowie die gewonnenen Erkenntnisse für den Prototyp zu beschreiben.

5.1.1 Rendern einer 3D-Datei (render-3d-file)

Zielsetzung Ziel dieses POC ist das Erproben von Three.js und GLB-Dateien. Dafür soll eine Testanwendung geschrieben werden, welche ein 3D-Modell im Web texturiert und animiert darstellen kann.

Durchführung Ein 3D-Modell, das aus zwei Würfeln besteht, wurde in Blender erstellt und als GLB-Datei, einschließlich verschiedener Materialien, exportiert.

Anschließend wurde eine Webanwendung geschrieben, welche die Grundfunktionen von Three.js implementiert. Es wird eine Three.js-Szene erstellt, die das 3D-Modell lädt und animiert auf der Webseite darstellt.

Ergebnisse Dem HTML-DOM wird ein Canvas-Element hinzugefügt. Dieses stellt die Inhalte der Szene dar, die aus der Sicht des Kameraobjekts über den Renderer gerendert werden. Dadurch kann das importierte 3D-Modell korrekt geladen, gerendert und animiert werden (vgl. Abb. 5). Zudem reagieren die Objekte korrekt auf eine Veränderung der Fenstergröße.

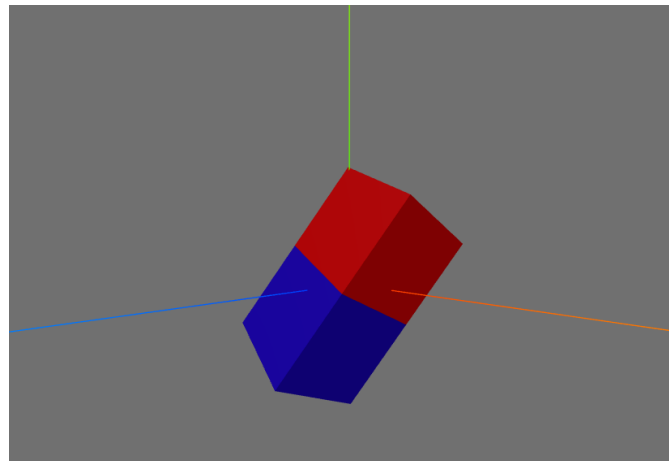


Abbildung 5: Das Canvas-Element beinhaltet zwei Würfel, welche in einem Koordinatensystem rotieren

(Quelle der Abbildung: POC "render-3d-file").

Bewertung Der POC zeigt, dass Blender, das GLB-Dateiformat, HTML und Three.js gut kombiniert werden können. Das Darstellen, Texturieren und Animieren von 3D-Modellen ist grundsätzlich möglich, sollte in Zukunft jedoch noch mit komplexeren Modellen und Funktionen getestet werden. Die Technologien und Ansätze sind für eine Nutzung im Prototyp geeignet.

5.1.2 Animation von 3D-Modellen über Namenskonvention (layer-naming-test)

Zielsetzung Entwicklung einer Namenskonvention, welche für die Animation von 3D-Modellen genutzt werden kann. Diese Namenskonvention soll ausgelesen werden und animierte Exploded Views auf vordefinierten Achsen ermöglichen.

Durchführung

1. Entwicklung einer Namenskonvention.
2. Umbenennung der Objekte im 3D-Modell.
3. Entwicklung einer Funktion, welche die Benennung der Objekte ausliest und auf Basis des Animationsfortschritts bewegen kann.
4. Entwicklung einer Interaktionsmöglichkeit für die Animation.

Ergebnisse Objekte können umbenannt werden, um die Explosionsebene und optional die Explosionsrichtung zu definieren. Objekte, die als explodierbar markiert sind und keine Explosionsrichtung haben, werden anhand der globalen Richtung animiert. Die Ebene stellt dabei den Abstand zur Mitte des 3D-Modells dar, welcher nach Abschluss der Animation erreicht wird (vgl. Abb. 6). Die möglichen Endpunkte sind vereinfacht als Kugeln dargestellt, in diesem POC jedoch auf die Schnittpunkte der X-,Y- und Z-Achsen mit den Kugeln begrenzt, da nur diese Achsen als Richtungen ausgewählt werden können. Zur Interaktion wurde ein Slider-Element entwickelt, das den Fortschritt der Animation flüssig steuern kann. Ebenfalls wurden OrbitControls zur Interaktion mit dem Modell hinzugefügt.

Format: `exp-L<Ebenenummer>-dir<Richtungscode>-<Name>`

exp-: (erforderlich) Markiert das Objekt als explodierbar.

L<Ebenenummer>: (erforderlich) Definiert die Explosionsebene (z. B. L1, L3).
Höhere Zahlen = größere Entfernung.

dir<Richtungscode>: (optional) Spezifische Explosionsrichtung.

`dir` direction

X, Y, Z Achse

POS/NEG Positiv/Negativ

Fallback: Wenn der Richtungscode fehlt, wird die positive Y-Achse genutzt.

<OptionalerName>: Beliebige Benennung der Teile.

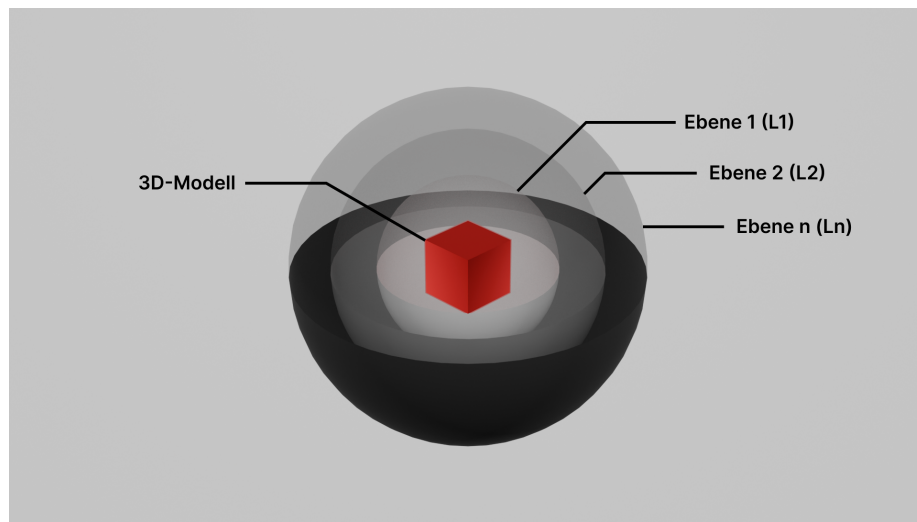


Abbildung 6: 3D Modell, welches von Ebenen umgeben wird, die Ebenen (L1 bis Ln) werden verwendet, um die Distanz zur Ausgangsposition festzulegen, welche nach Abschluss der Animation erreicht wird.

Beispiele:

- `exp-L1-globalBsp` (Ebene 1, globale Richtung)
- `exp-L1-dirYPOS-deckel` (Ebene 2, explodiert nach oben)
- `exp-L3-dirZPOS-beschriftung` (Ebene 3, explodiert zur Kamera nach Vorne)

Wichtig:

- Ohne `exp-` oder gültige `L<Nummer>` wird das Objekt nicht verarbeitet.
- Bei fehlendem `dir<Code>` wird die globale Richtung genutzt und eine Warnung ausgegeben.

Bewertung Aus dem POC hat sich ergeben, dass die Animation eines 3D-Modells auf vorgegebenen Achsen anhand einer Namenskonvention möglich ist. Die Richtungen und Ebenen ermöglichen verschiedene Anordnungen und Animationen. Die Usability ist jedoch ausbaufähig, da das Objekt zur Änderung der Animation neu benannt und exportiert werden muss. Besonders bei umfangreicheren 3D-Modellen und Animationen (vgl. Abb. 8) kann es notwendig sein, die Animation iterativ anzupassen. Dafür werden mehrere Exporte erforderlich. Komplexe Eltern-Kind-Beziehungen und andere Verschachtelungen können zu Problemen bei der Animation führen.

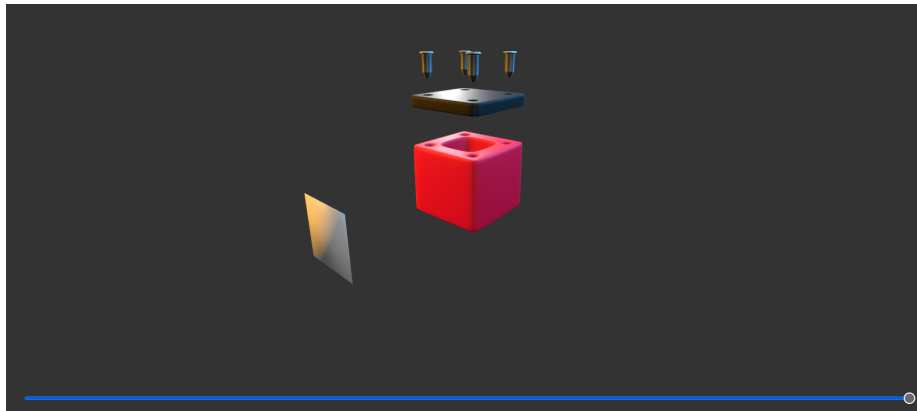


Abbildung 7: Beispielhaftes Ergebnis aus diesem Poc
(Quelle der Abbildung: POC "layer-naming-test").

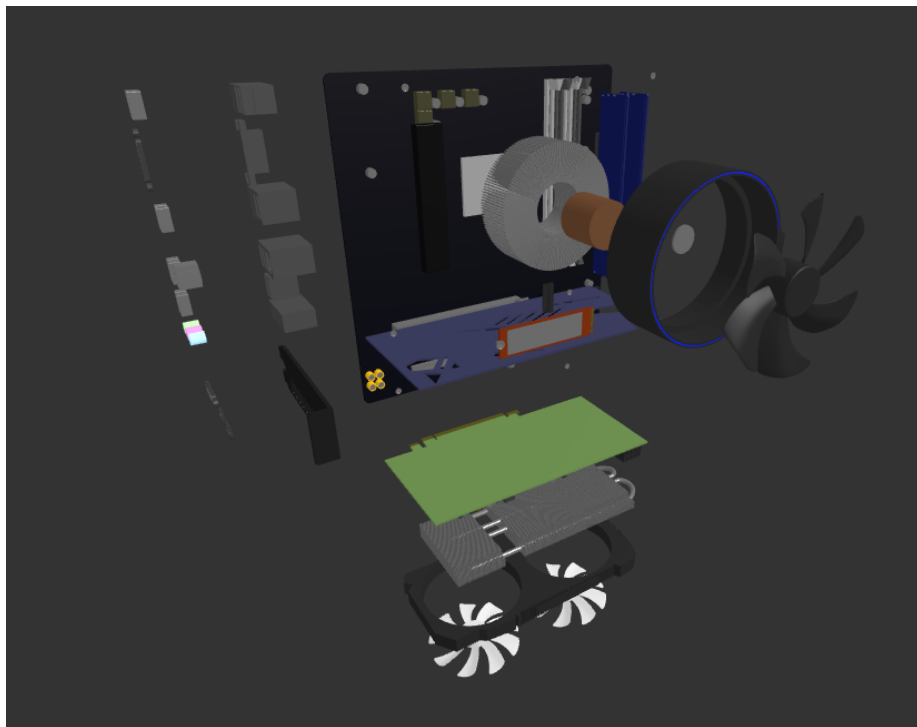


Abbildung 8: Beispielhaftes Ergebnis aus dem POC "complex-layer-naming-
test" mit einem komplexeren 3D Modell
(Quelle der Abbildung: POC "complex-layer-naming-test").

5.1.3 2D-Beschriftungselemente (text-labels-2d)

Zielsetzung Ziel war die Erprobung, wie Beschriftungselemente für einzelne Komponenten eines 3D-Modells generiert werden können. Diese Komponenten sollen neben dem dynamischen Inhalt auch eine passende Position im Verhältnis zum geklickten Objekt einnehmen und diesem folgen.

Durchführung

1. Erprobung eines Raycasters, zur Auswahl von Objekten.
2. Erstellen eines CSS2D-Renderers, der die 3D-Szene überlagert und HTML Elemente rendern kann.
3. Laden von Textinhalten aus einer Json-Datei in ein CSS2D-Objekt
4. Das 2D-Beschriftungselement im Overlay an das ausgewählte 3D-Objekt anhängen.

Ergebnisse Objekte können nun angeklickt werden und erhalten, wenn sie ausgewählt werden, ein HTML-Element im Overlay zugewiesen, welches sich mit dem 3D-Modell bei Animationen und Interaktionen bewegt. Dort können auf Basis der in 5.1.2 beschriebenen Namenskonvention Textinhalte aus einer Json-Datei geladen und angezeigt werden.

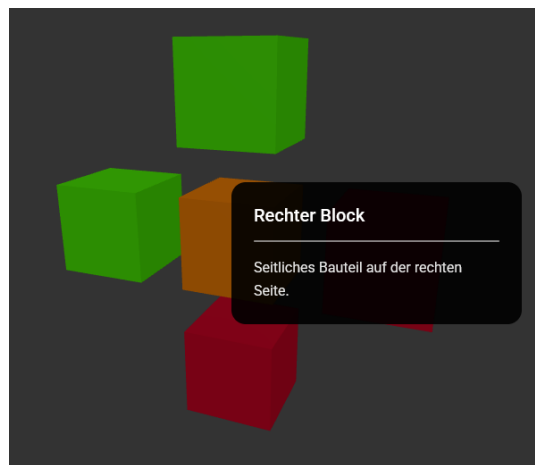


Abbildung 9: Beschriftungselement, welches beim klicken einer Komponente des 3D-Modells angezeigt wird
(Quelle der Abbildung: POC "text-labels-2d").

Bewertung Das Einfügen von Beschriftungselementen ist gut umsetzbar. Die Inhalte können übersichtlich in einer Json-Datei gespeichert werden und das Design der CSS2D-Objekte ist über CSS mit einer effizienten und Bekannten Technologie umsetzbar. Die Positionierung der Elemente ist jedoch ausbaufähig, da sie in einem Overlay über dem 3D-Modell gerendert werden. Dadurch überdecken sie häufig den ausgewählten Inhalt des 3D-Modells, den sie

beschreiben sollen. Zudem sollte im Prototyp eine robustere und intuitivere Implementation für das Öffnen und Schließen der Cards entwickelt werden. Aktuell kann es beim Nutzen der Orbit Controls zu unabsichtlichen Eingaben kommen. Es besteht die Möglichkeit, CSS2D-Objekte für andere Komponenten in diesem Projekt zu verwenden. Statt der Beschriftungselemente könnten sie auch als Trigger zum Öffnen anderer Beschriftungselemente verwendet werden. Es wäre möglich, für Objekte mit Textinhalten Buttons als CSS2D-Overlay einzublenken.

5.1.4 3D-Beschriftungselemente (text-labels-3d)

Zielsetzung Das Ziel dieses POCs war die Entwicklung eines Beschriftungselements, welches innerhalb der 3D-Umgebung dargestellt wird. Die Komponente soll ein Zeigeelement haben, das auf das ausgewählte 3D-Objekt zeigt.

Durchführung Zunächst wurden verschiedene Möglichkeiten erprobt, um den Text in den dreidimensionalen Raum einzubinden, darunter 3D-Texte und Texturen. Aufgrund des gewünschten Designs und der Leistung fiel die Auswahl auf Texturen. Für das Umsetzen des Zeigeelements und der Texte musste ein System entwickelt werden, welches den pointerCanvas positionieren kann und einen nahtlosen Übergang in den textBlockCanvas gewährleistet.

Ergebnisse Wenn ein 3D-Objekt ausgewählt wird, werden die passenden Texte geladen und an dieses Objekt wird ein texturiertes Element angehängt. Die Textur wird aus zwei Canvaselementen, dem pointerCanvas (blau) und dem textBlockCanvas (grün) (vgl. Abb. 11), zusammengesetzt. Der pointerCanvas kann auf der rechten oder linken Seite angebracht werden, um den Text zu positionieren und auf das ausgewählte Objekt zu zeigen. Zudem gibt es die Möglichkeit, über ein Menü die Beschriftungselemente um $\pm 90^\circ$ oder 45° gedreht anzuzeigen (vgl. Abb. 12).

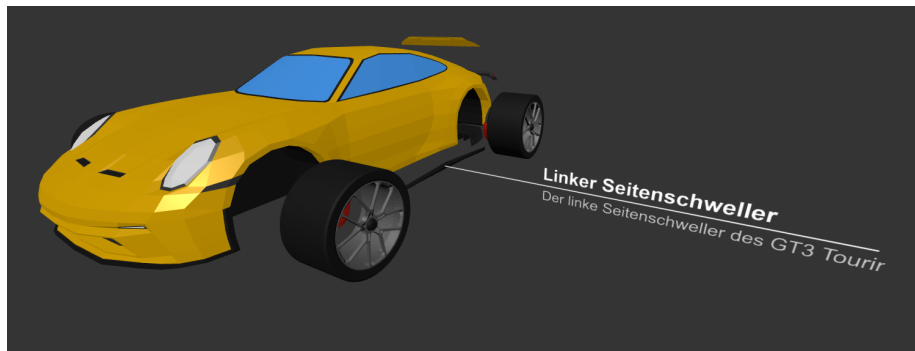


Abbildung 10: Beschriftungselement "pointer", welches beim klicken einer Komponente des 3D-Modells angezeigt wird
(Quelle der Abbildung: POC "text-labels-3d").

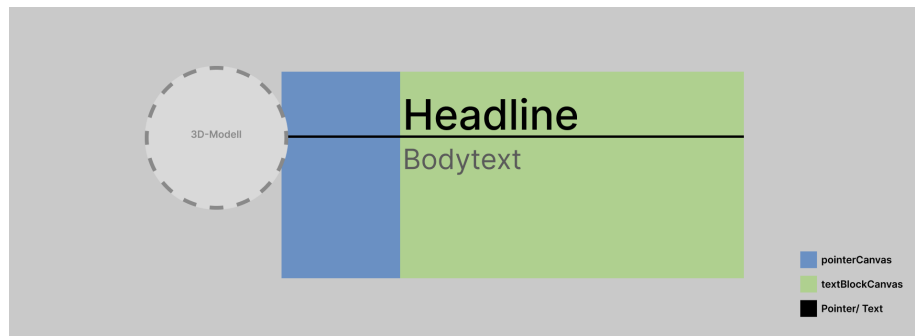


Abbildung 11: Das Beschriftungselement "pointer", besteht aus zwei Canvas-elementen, dem pointerCanvas und dem textBlockCanvas. Der pointerCanvas kann links oder rechts angebracht werden.

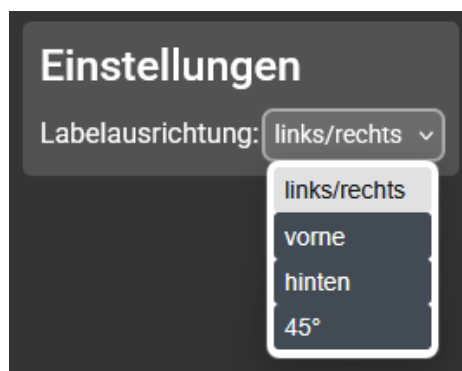


Abbildung 12: Das Menü bietet einige Einstellungen für die Positionierung des Beschriftungselements "pointer".

(Quelle der Abbildung: POC "text-labels-3d").

Bewertung Der POC konnte erfolgreich abgeschlossen werden. Es ist möglich, ein Beschriftungselement in die Szene einzufügen und dieses mit Inhalten zu füllen und dynamisch zu positionieren. Es wurden jedoch auch einige Problemstellungen erkannt. Das Label zählt als 3D-Objekt und kann geklickt werden. Dadurch kann unerwünschtes Verhalten beim öffnen und schließen entstehen. Zudem kann das Element bei komplexeren Szenen mit den animierten Objekten kollidieren. Dadurch wird der Text von anderen 3D-Objekten verdeckt. Dieses Problem hängt vom verwendeten Modell ab und kann teilweise mit den Optionen zur Rotation (vgl. Abb. 12) verhindert werden. Ein weiteres Problem bei diesem Ansatz ist die Verwendung von Texturen. Sie ermöglichen es, das Design umzusetzen und sind beidseitig lesbar. Sie haben jedoch keine Textverarbeitungsfunktionen wie Umbrüche oder Auflistungen integriert. Es müssen also Funktionen und Verbesserungen entwickelt werden, um diesen POC in den Prototyp aufnehmen zu können.

5.1.5 2D-Beschriftungselemente mit Hervorhebung (text-label-highlight-card)

Zielsetzung Es soll ein Beschriftungselement entwickelt werden, das unabhängig vom 3D-Modell platziert werden kann. Dabei muss eine Methode entwickelt werden, welche die ausgewählte Komponente des 3D-Modells hervorhebt, um die Verbindung zwischen Text und Objekt zu visualisieren.

Durchführung

1. Gestalten der Card-Komponente.
2. Implementieren und animieren der Card.
3. Erprobung verschiedener Methoden zur Hervorhebung der ausgewählten Komponente.

Ergebnisse Bei der Gestaltung der Card wurden weitere Inhalte, wie beispielsweise Listen, berücksichtigt (vgl. Abb. 13). Zudem wurde ein Light- und Darkmode implementiert, welcher über eine Tweakpane-GUI (vgl. 5.1.6) gesteuert werden kann. Die Card konnte mit Animejs aus POC 5.1.7 erfolgreich animiert werden. Um die ausgewählte Komponente aus dem 3D-Modell hervorzuheben, wurden verschiedene Optionen erprobt (vgl. Abb. 14).

Bewertung Der POC war erfolgreich und stellt unter den erprobten Optionen die vielseitigste Möglichkeit zur Beschriftung von Komponenten dar. Die fest positionierten Cards weisen einige Vorteile auf. Sie sind im Vergleich am besten mit schmalen und kleinen Bildschirmen kompatibel. Der Textinhalt ist immer lesbar und das 3D-Modell wird nur selten überdeckt. Im Vergleich zu anderen Komponenten wie dem Pointer-Element können Nutzer diese Option einfacher umgestalten, da sie mit HTML und CSS erstellt ist. HTML-Inhalte sind zudem besser mit Screenreadern und anderen Tools für erweiterte Barrierefreiheit kompatibel.

Zum Hervorheben der ausgewählten Komponente bieten sich mehrere Optionen an (vgl. Abb. 14). Dabei hat sich ergeben, dass das ausgewählte 3D-Objekt nicht modifiziert werden sollte. Da diese Darstellung, während das Objekt im

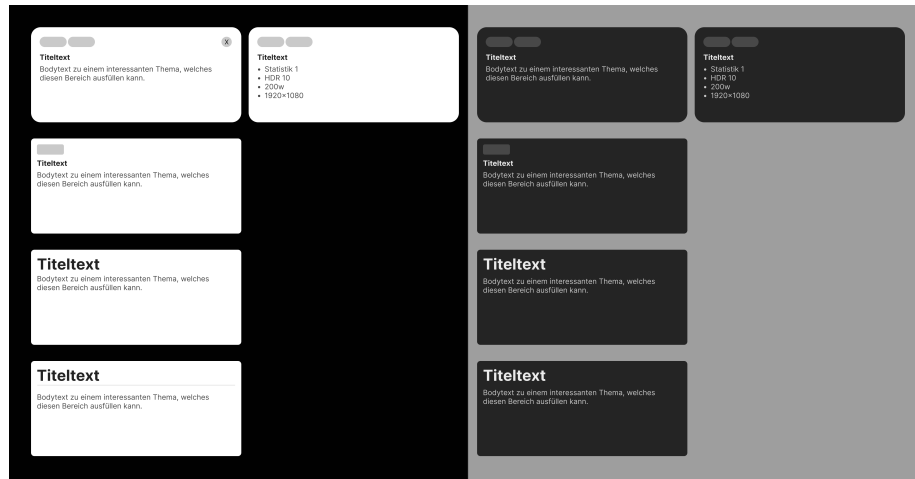


Abbildung 13: Das Design der Card-Elemente, in verschiedenen Varianten.

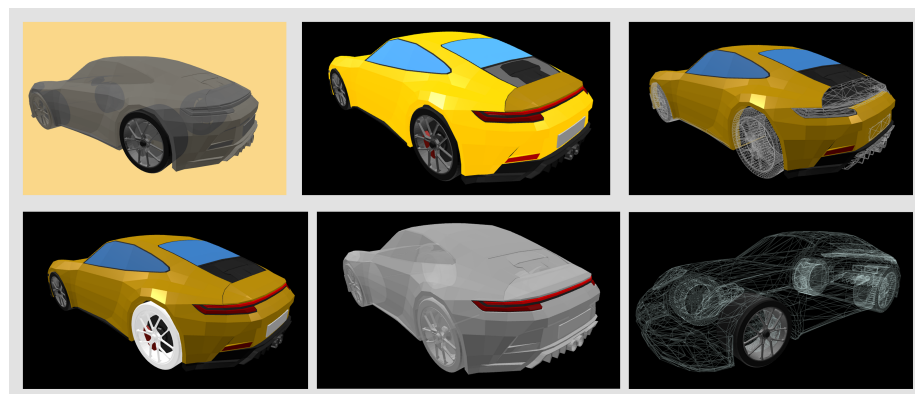


Abbildung 14: Eine Auswahl der besten Optionen zur Hervorhebung der ausgewählten Komponente.

(Quelle der Abbildung: POC "text-label-highlight-card").

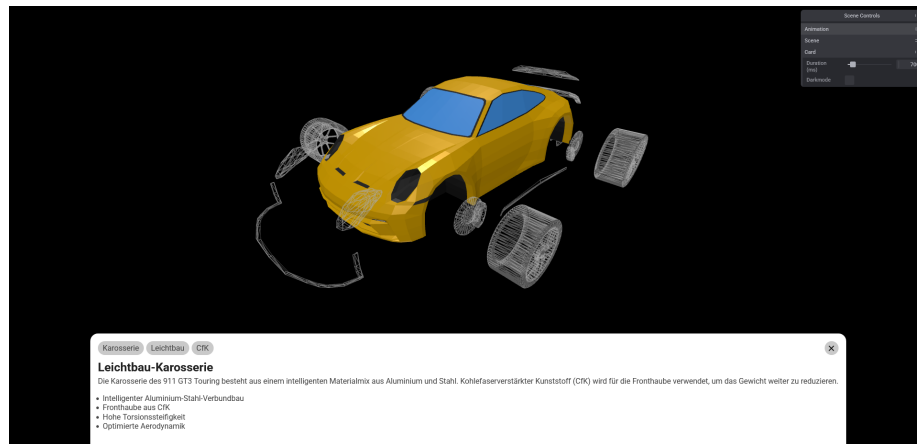


Abbildung 15: Hervorhebung der ausgewählten Komponente und anzeige der Beschriftung in einer Card am unteren Bildschirmrand.

(Quelle der Abbildung: POC "text-label-highlight-card").

Fokus liegt und betrachtet wird, die Realität verfälschen würde. Daher sollten alle anderen Objekte an Prägnanz verlieren. Diese Hierarchie kann durch das abschwächen oder entfernen der Farb- und Transparenzeffekte ermöglicht werden. Eine weitere Option zur Visualisierung sind Wireframes, welche den 3D-Kontext ausschöpfen und die umliegenden Objekte teilweise durchsichtig machen. Sie ermöglichen neben der Exploded View einen weiteren Einblick in das 3D-Modell (vgl. Abb. 14) und eignen sich besonders für Objekte, die keine prägnante Farbgebung haben.

5.1.6 Interaktives Menü (tweakpane-GUI)

Zielsetzung Mit diesem POC sollte eine GUI Bibliothek ausgewählt und erprobt werden, welche zum testen der Funktionalität und zum schnellen Experimentieren mit verschiedenen Parametern eingesetzt wird, da Eigenentwicklungen (vgl. 12) Mehraufwand und Probleme bei Design und Funktionalität geboten haben.

Durchführung Es wurden diverse Bibliotheken untersucht und getestet. Dazu zählen Tweakpane (Kokubun, 2024), lil-gui (Georgealways, 2024) und control-panel (Freeman, 2019).

Ergebnisse Tweakpane wurde für einen tiefgreifenden Test ausgewählt. Dabei wurden verschiedene Datentypen wie Farben, Zahlen, Koordinaten usw. getestet. Zudem wurden Werte aus Konfigurationsdateien gelesen und live in der 3D-Szene Angewendet. Dadurch konnten Szene, Kamera und Lichter zur Laufzeit bearbeitet werden.

Bewertung Es konnte erfolgreich eine GUI-Bibliothek ausgewählt werden. Tweakpane wurde ausgewählt, da es auf weitere Abhängigkeiten verzichtet und



Abbildung 16: Verschiedene UI Komponenten
(Quelle der Abbildungen: lil-gui (Georgealways, 2024), Tweakpane (Kokubun, 2024), und control-panel (Freeman, 2019)).

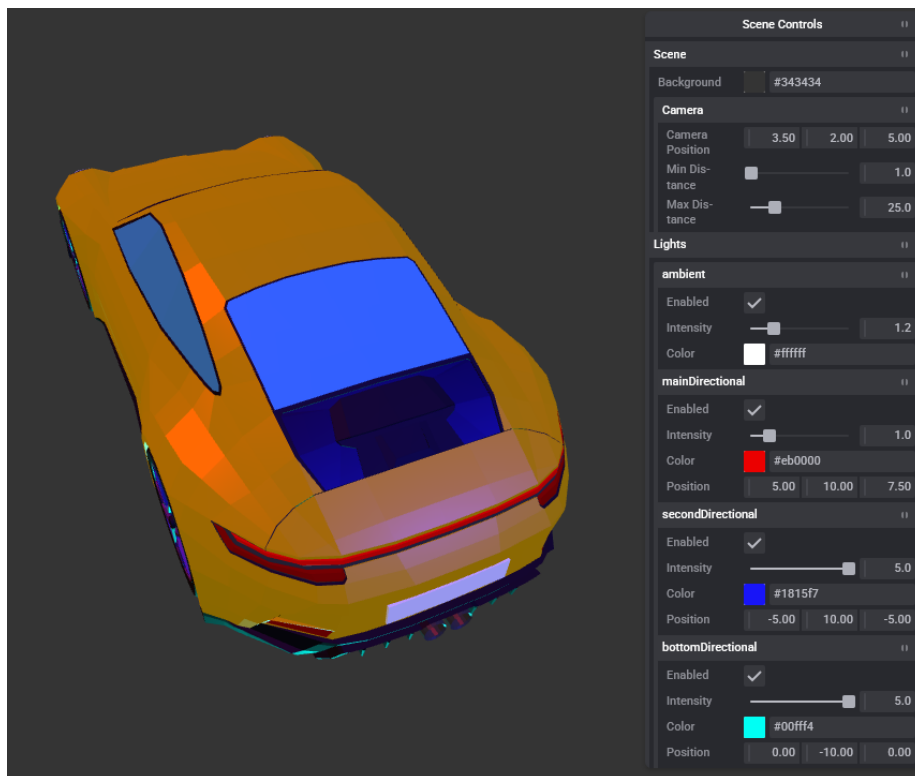


Abbildung 17: Tweakpane Anwendung, Manipulation der Lichter
(Quelle der Abbildung: POC "tweakpane-GUI").

aktiv weiterentwickelt wird. Zudem bietet Tweakpane Funktionen wie Ordner und Farbwähler und kann dank der ausführlichen Dokumentation auch durch Plugins und eigene Designs erweitert werden (vgl. (Kokubun, 2024)). Durch diesen POC können nun zur Laufzeit diverse Werte erprobt werden und die Nutzererfahrung wurde verbessert.

5.1.7 Animationsbibliothek (animejs-test)

Zielsetzung In diesem POC sollte eine Animationsbibliothek erprobt werden, welche für die Animation von HTML- und CSS-Komponenten sowie Zahlenwerten für die Exploded View Animation verwendet werden kann.

Durchführung Zunächst wurden in der Recherche einige Animationsbibliotheken verglichen. Eine der Optionen wurde für weitere Tests ausgewählt und implementiert.

Ergebnisse Animejs hat sich als gute Option herausgestellt und wurde implementiert. Es wurden HTML- und CSS-Elemente animiert. Zudem wurde das Zusammenspiel mit Tweakpane und Json-Daten aus einer Konfigurationsdatei getestet.

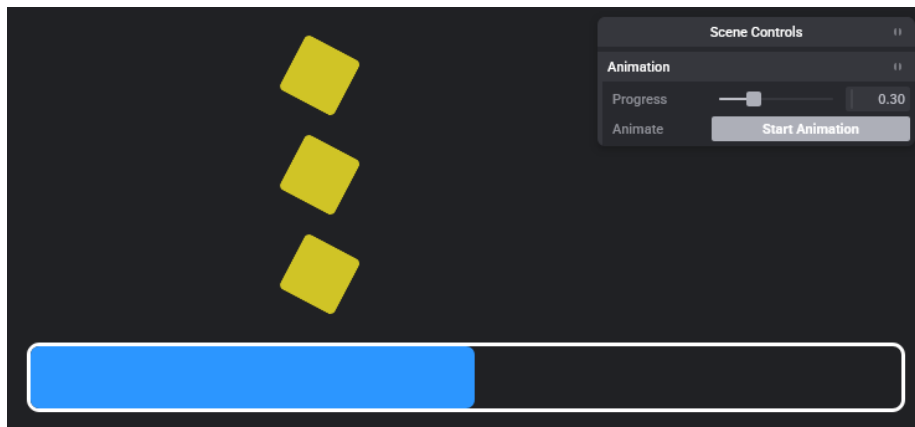


Abbildung 18: Rotierende Quadrate, welche sich von links nach rechts bewegen. Ein Quader, welcher Blau gefüllt wird und Tweakpane UI (Quelle der Abbildung: POC "tweakpane-GUI").

Bewertung Der POC war erfolgreich. Es ist sowohl möglich, HTML- und CSS-Komponenten zu animieren als auch Werte aus einer Konfigurationsdatei zu manipulieren. Die Animation kann auch erfolgreich über Tweakpane UI Komponenten gesteuert werden.

5.2 Prototyp

Die in den beschriebenen POCs gewonnenen Erkenntnisse wurden dann in den Prototyp integriert. Dabei wurden die erprobten Funktionen weiter ausgearbei-

tet und erweitert und die Code-Qualität sowie die Nutzerfreundlichkeit wurden angehoben. Im folgenden Abschnitt wird der Entwicklungsprozess vom POC Ergebnis zur Finalen Komponente und dem Gesamtbild beschrieben.

5.2.1 3D-Darstellung

1. Grundlagen der 3D-Darstellung im Web Die Technische Grundlage zur 3D-Darstellung bildet weiterhin WebGL, mit Three.js als Bibliothek zur Abstraktion, da deren Vorteile (vgl. 4.4.2) in verschiedenen POCs erprobt werden konnten.

2. Laden und Verarbeiten eines 3D-Modells Im Prototyp wird weiterhin das GLB-Dateiformat genutzt, da die bekannten Vorteile (vgl. 4.4.3) in den POCs erprobt und Bestätigt werden konnten.

Eine wichtige Neuerung im POC ist nun jedoch die Vorbereitung des 3D-Modells. Es wird auf die Komplexe Namenskonvention aus 5.1.2 verzichtet. Stattdessen muss jedes 3D-Objekt, welches animiert oder beschriftet werden soll, einen eindeutigen Namen erhalten.

Beim Laden des 3D-Modells wird auf die Baumstruktur zugegriffen, welche alle 3D-Objekte der Szene enthält. Dabei wird auf die erste Ebene der Parent-Objekte zugegriffen (vgl. Abb.19). Diese Objekte können unabhängig voneinander ausgewählt und hervorgehoben werden. Wenn eines dieser Objekte weitere Kindelemente hat, können diese separat animiert werden. Beim Auswählen und Hervorheben zählen sie jedoch immer als eine Gruppe (vgl. Abb.20). Diese Designentscheidung wurde getroffen, damit Objekte, wie beispielsweise Schrauben, als Gruppe hervorgehoben werden und eine zentrale Beschriftung erhalten können. Die Animation kann jedoch trotzdem an verschiedenen Positionen in unterschiedliche Richtungen starten.

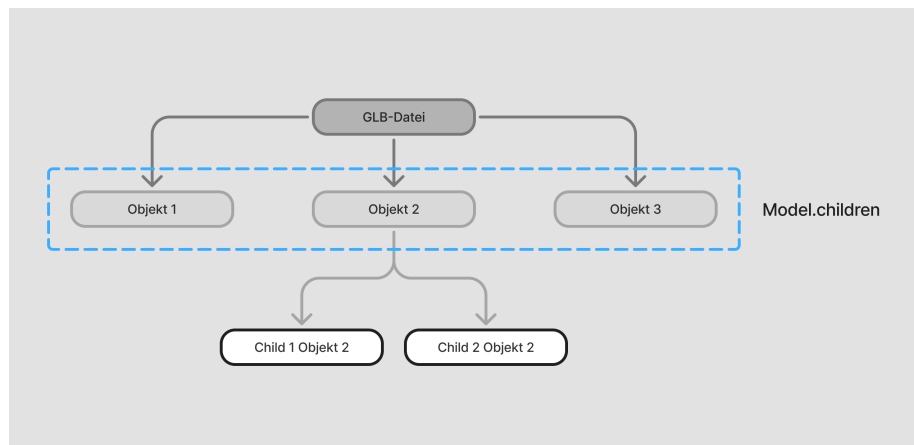


Abbildung 19: Visualisierung der Baumstruktur mit Eltern-Kind-Beziehungen im 3D-Kontext.

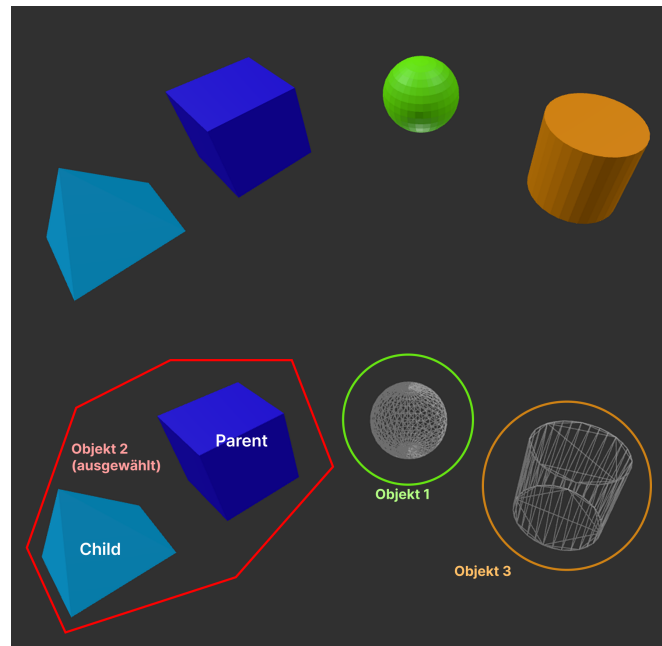


Abbildung 20: Visualisierung der Eltern Kind Struktur bei der Objektauswahl.
3D Darstellung von Abb. 19

5.2.2 Beschriftungselemente

Wie in 5.2.1 und 4.3 beschrieben, gibt es bei der Beschriftung von 3D-Modellen diverse Möglichkeiten und Problemstellungen. Im Prototyp wurden alle in den POCs erprobten Ansätze zu Beschriftungselementen implementiert, verbessert und als Optionen angeboten. Der Wechsel erfolgt über den Wert `infoElementType` in der Konfiguration. Zur Verbesserung der Code-Qualität wurde im Prototyp die Klasse `InfoElementHandler` als Basis für alle Beschriftungselemente angelegt. Diese legt die Grundlegenden Funktionen fest und ermöglicht eine saubere Implementierung für alle Elemente im `ClickHandler`. Alle Infoelemente bleiben nun während der Verwendung der Orbitcontrols geöffnet und schließen sich nur, wenn erneut auf ein Objekt, ein anderes Element oder ein GUI-Element zum Schließen geklickt wird. Die Daten für die Infoelemente werden in einer Json-Datei im folgenden Format gespeichert.

```
{
  "Objekt-Name": {
    "title": "Card-Titel",
    "body": "Body-Text",
    "pills": ["Pill1", "Pill2", "Pill3"],
    "list": [
      "Eintrag 1",
      "Eintrag 2",
      "Eintrag 3"
    ]
  }
},
```

```
}
```

Pointer Das in `pointer-handler` implementierte Infoelement hat den Aufbau aus Abb.11 beibehalten, die möglichen Inhalte und Optionen wurden jedoch ausgeweitet. In der `pointerConfig` können Werte für die Seite des `pointerCanvas` ausgewählt werden. Dadurch wird die Beschriftung entweder immer rechts oder immer links vom ausgewählten Objekt positioniert. Die Option `auto` ermöglicht es alternativ, dass die Seite dynamisch auf Basis der Kameraposition gewählt wird. Dadurch werden Kollisionen mit dem Objekt verringert. `rotationY` ermöglicht es ebenfalls, Kollisionen und Überlagerungen zu vermeiden. Dadurch kann das Pointerelement entweder automatisch auf die Kamera ausgerichtet werden. Alternativ kann eine feste Rotation um die Y-Achse implementiert werden und die Farbgebung ist nun konfigurierbar. Da Überlagerungen jedoch nicht vollständig ausgeschlossen sind, werden die Elemente nun über dem 3D-Modell gerendert. Auch diese Option kann, abhängig von den Einstellungen bei der Hervorhebung und der Farbgebung des Pointerelements und des 3D-Modells, jedoch für Kontrastprobleme und schlechte Lesbarkeit sorgen (vgl. Abb. 21).

Diese Implementation ist problematisch, wenn Gruppen ausgewählt werden. In diesen Situationen zeigt das Pointerelement auf den Mittelpunkt des Objekts bzw. der Gruppe. In solchen Situationen sollte die Hervorhebung aktiviert werden, oder eine alternative Beschriftung gewählt werden. Die Textverarbeitungsfunktionen wurden in dieser Iteration deutlich verbessert. durch `maxWidth` kann die Skalierung angepasst werden. Auch Textumbrüche sowie Aufzählungen sind nun möglich.

```
pointerConfig: {
  defaultSide: 'auto',
  rotationY: '0',
  maxWidth: 1800,
  titleColor: "#ffffff",
  bodyColor: "#999999",
  lineColor: "#ebebeb"
},
```

Bottom Card Es gibt nun ein einheitliches Design für die zweidimensionalen Infoelemente. Dabei wurde das Design aus 5.1.5 aufgegriffen und verbessert (vgl. Abb. 22). Beim öffnen und schließen der Card wird nun eine Animation abgespielt, die über die Optionen konfiguriert werden kann.

```
cardConfig: {
  animationDuration: 700,
  isDarkmode: true
},
```

Attached Card Diese Card übernimmt die Technologie und Funktionalität aus dem POC 5.1.3, erweitert diese jedoch um das Design und den Funktionen der Bottom Card. Dadurch sind weitere Inhalte wie Pills, Listen und ein Button zum schließen der Card hinzugefügt worden.



Abbildung 21: Pointerelement mit beispielhafter Farbgebung und neuer Darstellung mit Überlagerung.

5.2.3 Interaktionen

Bei der Interaktion wurden ebenfalls mehrere Möglichkeiten implementiert. Die Animation kann über einen Slider präzise gesteuert werden. Alternativ kann der Nutzer die Animation über einen Button starten, Pausieren und auch rückwärts ablaufen lassen. Diese Implementation ist am Vielseitigsten, da sie auch auf mobilen Geräten funktioniert und einen flüssigen Ablauf gewährleistet. Zudem ist es möglich, die Animation über das Scrollrad der Maus oder ein Touchpad zu steuern. Bei dieser Option ist ein easing implementiert, trotzdem führen Mausräder mit Freilauf und Touchpads zu besseren Animationen.

5.2.4 Animation

Die Animationslogik wurde im Prototyp stark überarbeitet und im Zuge der Modularisierung in eine eigene Klasse ausgelagert. Die Klasse `AnimationHandler` bündelt nun alle Funktionen zur Steuerung und Aktualisierung der Animation. Die Verwaltung der Animation erfolgt nun wie beschrieben über den Namen des 3D-Objekts, welcher mit einem Eintrag in der `exp-config.json` übereinstimmen muss. Ein Objekt kann die Parameter `level`, `expDirection`, `sequence` und `speedMultiplier` haben. Hier ein Auszug:

```
{
  "_comment": "[X|Y|Z]",
  "objects": {
    "main-body": {
      "level": 0,
      "expDirection": [0, 0, 0]
    },
    "Diffusor": {
      "level": 1.5,
      "expDirection": [0, 0, 1],
    }
  }
}
```

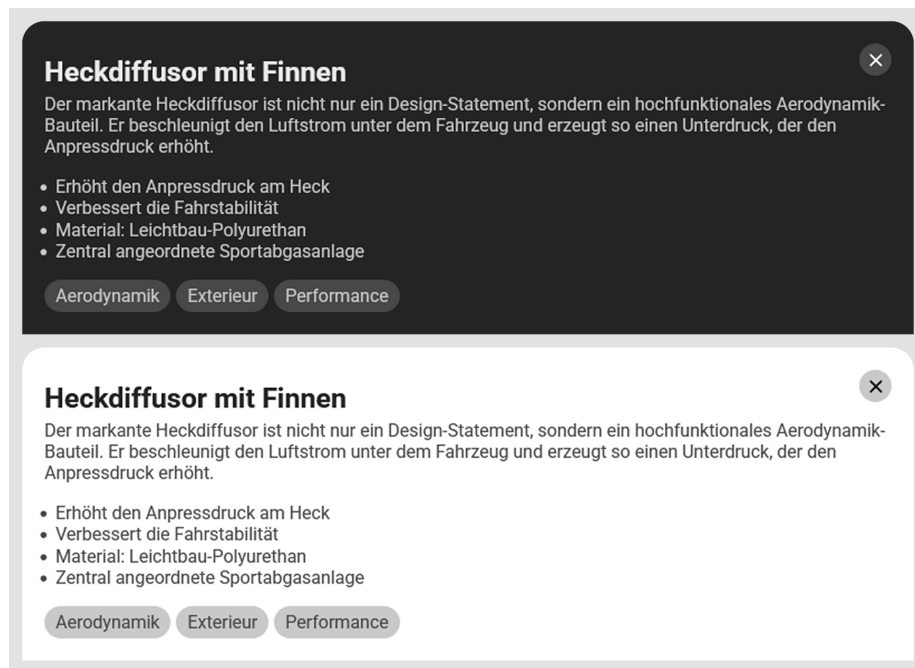


Abbildung 22: Bottom Card im Dark- und Lightmode
(Quelle der Abbildung: Prototyp).

```

    "sequence": 2,
    "speedMultiplier": 1.5
  },
  "Kennzeichen": {
    "level": 3,
    "expDirection": [-1, 0, 0],
    "sequence": 1,
    "speedMultiplier": 2
  }
}
}

```

Parameter:

- **level:** bestimmt, welche Distanz das Objekt von der Ausgangsposition bis zum Abschluss der Animation zurücklegt (vgl. Abb.23). Dabei können auch Dezimalwerte ausgewählt werden, um die Distanz präzise festlegen zu können.
- **expDirection:** legt die Bewegungsrichtung als Vektor [X, Y, Z] fest. Die Länge des Vektors ist irrelevant, da er normalisiert wird. Dadurch können Nutzer die genaue Richtung angeben und Kollisionen während der Animation vermeiden.

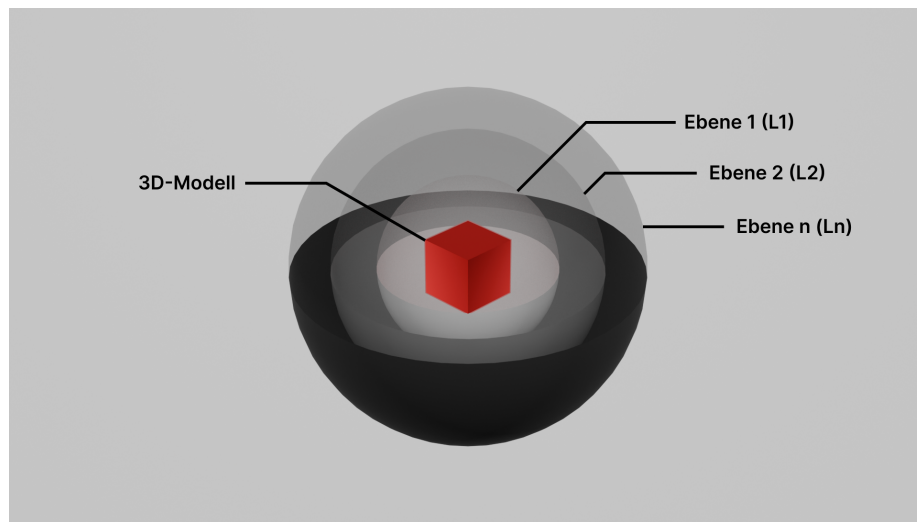


Abbildung 23: 3D Modell, welches von Ebenen umgeben wird, die Ebenen (L1 bis Ln) werden verwendet, um die Distanz zur Ausgangsposition festzulegen, welche nach Abschluss der Animation erreicht wird.

- **sequence**: falls **useSequenceAnim: true** konfiguriert wurde, können Objekte in bestimmten Sequenzen animiert werden. Dabei wird die Animationszeit in gleich große Intervalle unterteilt. Alle Objekte werden dann nur während der zugewiesenen Sequenz bewegt. In der letzten Sequenz ($(\text{maxSequence} + 1)$) werden dann alle Objekte ohne zugewiesene Sequenz animiert (vgl. Abb.24). Für diese Option muss der Algorithmus zur Animation geändert werden. Statt die Distanz über $\text{Zielebene} * \text{Ebenenabstand} * \text{Animationsfortschritt}$ zu berechnen, müssen für jedes Objekt der Start- und Endpunkt der Animationssequenz (**progressStart** und **progressEnd**) berücksichtigt werden. Dafür wird jeweils eine Lineare Funktion erstellt, welche den globalen Animationsfortschritt **expFactor** auf das aktuelle Intervall abbildet. Da die Objekte bei dieser Variante sich in kürzeren Zeiträumen über größere Distanzen bewegen müssen, sollte eine höhere **animationDuration** verwendet werden.

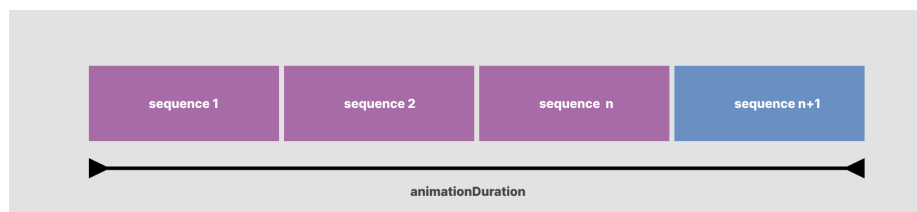


Abbildung 24: Aufteilung der Sequenzen auf die Animationsdauer

- **speedMultiplier** beeinflusst die Animationsgeschwindigkeit eines Ob-

jekts, indem der `expFactor` mit dem `speedMultiplier` verrechnet wird. Dadurch kann ein Objekt schneller das angegebene Ziel beziehungsweise die angegebene Ebene erreichen. Diese Option ermöglicht komplexere Animationen, welche interessanter, flüssiger und natürlicher wirken.

5.2.5 Modularisierung

Wie bereits in den vorherigen Abschnitten beschrieben, wurden zur Verbesserung der Wartbarkeit und um Erweiterungen besser zu ermöglichen, die verschiedenen Funktionen in einzelne Klassen ausgelagert. Dadurch wird die Entwicklung übersichtlicher und der Status von Variablen und Objekten kann besser verfolgt werden. Bei der Einbindung des npm-Pakets sind die Angabe der relevanten Dateien und Optionen sowie das Aufrufen der `ExplodedViewer`-Klasse ausreichend, um die Animation auf einer Webseite einzubetten. Dafür wurde eine Installationsanleitung angelegt.

5.2.6 Dateneinbindung und Nutzung

Die Daten für die Animation und die Beschriftungselemente sind, wie beschrieben, nun in entsprechende Json-Dateien ausgelagert. Die Konfiguration wurde ebenfalls überarbeitet. Entwickler geben wichtige Einstellungen in der `default-options.js` an. Nutzer des Pakets können diese Standardkonfiguration dann in einer `scene-config.json` oder in den `expOptions` innerhalb des Konstruktoraufrufs des `ExplodedViewer` überschreiben.

5.2.7 Zusammenfassung und Bewertung

Zusammenfassend war der Prototyp ein Erfolg. Die in den POCs erprobten Funktionen konnten ausgearbeitet, kombiniert und erweitert werden. Dazu zählen unter anderem eine umfangreiche Umstrukturierung der Codebasis, eine große Erweiterung der Exploded View Animationsfunktionen und die Verbesserung des Designs. Diese Funktionen konnten in Kombination mit Anpassungen der Codequalität, Modularisierung und Verbesserungen bei der Konfiguration die Nutzererfahrung sowohl auf Seiten der Entwickler als auch der Endnutzer deutlich verbessern. Anschließend konnte der Prototyp als npm-Paket gebaut und veröffentlicht werden. Dieses Paket wurde in einem externen Projekt erfolgreich integriert und getestet. Dies bestätigt die Wiederverwendbarkeit und die Praxistauglichkeit des Projekts.

6 Ergebnisse und Diskussion

6.1 Bewertung

Die Projektarbeit hat viele Erkenntnisse und einen funktionsfähigen Prototyp hervorgebracht. Es konnten Vorteile und Anwendungsfelder interaktiver Komponenten und Exploded Views im Web festgestellt werden. Des weiteren konnte ein Prototyp entwickelt und getestet werden, welcher das Erstellen interaktiver Exploded Views mit verschiedenen Animationen, Interaktionen und Beschriftungskomponenten ermöglicht. Dafür wurden verschiedene Technologien verglichen und erprobt, unterschiedliche Ansätze entwickelt und getestet sowie Herausforderungen überwunden.

6.2 Limitationen

Das Projekt hat gezeigt, dass die Dreidimensionalität viele Herausforderungen, aber auch viele Möglichkeiten bietet. Im Prototyp werden bereits viele Optionen für die Einbettung, Animation und die Darstellung angeboten. Die verfügbaren Funktionen und Optionen könnten jedoch ausgeweitet werden, um das Erstellen von noch umfangreicheren und interessanteren Animationen zu ermöglichen. Zudem könnten Nutzer die Ergebnisse dadurch weiter in das bestehende Design einer Webseite einbinden.

Neben dem Funktionsumfang könnte auch die Nutzererfahrung verbessert werden. Die Verwaltung der Animationen und Beschriftungen im Json-Format ist akzeptabel. Sie ermöglicht präzise Anpassungen und bietet eine menschen- und maschinenlesbare Struktur. Zudem können Dateien wiederverwendet, ausgetauscht oder KI-generiert werden. Besonders bei der Generierung von Animationen wäre eine Grafische Benutzeroberfläche für erste Entwürfe jedoch vorteilhaft, da die Animation so visualisiert wird. Folglich müsste der Nutzer weniger testen und sich weniger komplexe Umstände vorstellen.

Bei sehr umfangreichen Szenen kann es theoretisch durch das Traversieren der Elemente und das live Rendern im Browser zu Leistungseinbrüchen oder verlängerten Ladezeiten kommen. Die Ergebnisse hängen von der Hardware des Endnutzers ab, da alle Inhalte über JavaScript im Browser ausgeführt werden. Zudem variiert je nach Texturierung und Umfang des Modells die Ladezeit und der Speicherverbrauch. Es ist ratsam, die Ergebnisse auf den erwarteten Endgeräten zu testen und die 3D-Modelle zu optimieren. Bei Leistungstests in einigen Testszenen auf mobilen Geräten wie Smartphones und Laptops konnte eine stabile Leistung festgestellt werden. Dabei wurde eine Bildwiederholrate von weit über 30FPS erreicht (vgl. Kellner (2018))

7 Fazit und Ausblick

7.1 Zusammenfassende Beantwortung der Forschungsfragen

- **F1:** Wie lassen sich Exploded Views im Web modular und wiederverwendbar umsetzen und welche Best Practices sowie Herausforderungen existieren dabei?

Zur Beantwortung der Forschungsfrage wurden verschiedene Technologien und Lösungsansätze erprobt. Dabei traten Herausforderungen auf, welche unter Berücksichtigung der gewonnenen Erkenntnisse sowie der recherchierten Best Practices gelöst wurden. Das Ergebnis ist eine Modularisierte Code-Basis, welche über viele Funktionen verfügt. Nutzer können das Paket über npm installieren und umfangreiche Konfigurationen vornehmen.

- **F2: Welche Vorteile und Möglichkeiten bieten 3D Modelle und Exploded Views im Hinblick auf Storytelling?**

Die Vorteile von Exploded Views im Hinblick auf Storytelling konnten auf Grundlage von Studien und Recherche zu Exploded Views in anderen Domänen erforscht werden. Sie bieten dem Endnutzer die Möglichkeit, das Objekt besser zu betrachten und einen Einblick in die Einzelnen Komponenten zu erhalten. Durch diese intensive Interaktion kann der Nutzer das Objekt besser verstehen, Zusammenhänge erlernen oder andere Inhalte eigenständig entdecken. Abhängig vom 3D-Modell und der erstellten Animation kann so ein immersives Erlebnis geschaffen werden, das den Aufbau, die Entwicklung, die Produktion oder den nötigen Kontext zum Modell vermittelt. Die statische Darstellung eines Objektes wird so zu einer visuellen Erzählung.

- **F3: Wie lassen sich Interaktivität und Animation (z.B. Scroll- oder Hover-Effekte) integrieren?**

Zur Beantwortung der Forschungsfrage wurden verschiedene Optionen erprobt und deren Vor- und Nachteile verglichen. Interaktive Elemente benötigen im 3D-Kontext oft Raycaster, welche es ermöglichen, dass einzelne Komponenten des 3D-Modells ausgewählt werden. Für die Interaktion mit dem Gesamtmodell sollten Orbitcontrolls verwendet werden, welche eine natürliche und intuitive Form der Steuerung bieten. Für weitere Interaktionen und Animationen können oft CSS und JavaScript in Kombination mit JavaScript Event Listeners verwendet werden. Mit externen Bibliotheken wie Animejs kann der Funktionsumfang und die Qualität der Animationen jedoch gesteigert werden.

7.2 Ausblick

Zukünftige Projekte können auf den Erkenntnissen und Ergebnissen dieses Projekts basieren. Dabei könnten die angesprochenen Limitationen reduziert werden. Beispielsweise könnte ein Service entwickelt werden, welcher die erste Konfiguration durch ein grafisches Interface erleichtert oder durch einen Algorithmus zur automatischen Verarbeitung ersetzt. Des weiteren könnten Alternativen zur 3D-Einbindung erprobt werden. Vorbereitete, statische 2D-Inhalte könnten die Grafik und Ladezeiten verbessern, erzeugen jedoch potenziell mehr Datenverkehr und bieten weniger Möglichkeiten zur Interaktion. Diese Herausforderungen gilt es jedoch zu erforschen, zu lösen und zu vergleichen.

Literatur

- Apple. (2025). Human Interface Guidelines. Verfügbar 21. August 2025 unter <https://developer.apple.com/design/human-interface-guidelines/designing-for-ios/>
- Can I Use. (2025). *WebGL Support*. Verfügbar 3. September 2025 unter <https://caniuse.com/?search=webgl>
- Freeman, J. (2019). *control-panel: embeddable panel of inputs for parameter setting*. Verfügbar 11. Oktober 2025 unter <https://github.com/freeman-lab/control-panel>
- Garnier, J. (2025). *Animejs documentation*. Verfügbar 15. September 2025 unter <https://animejs.com/documentation/>
- Georgealways. (2024). *lil-gui: Makes a floating panel for controllers on the web. Works as a drop-in replacement for dat.gui in most projects*. lil-gui. Verfügbar 11. Oktober 2025 unter <https://github.com/georgealways/lil-gui>
- Google. (2014). Material Design 2 Guidelines. Verfügbar 21. August 2025 unter <https://m2.material.io/design/guidelines-overview>
- Halpern, M., Zhu, Y., & Reddi, V. J. (2016). Mobile CPU's rise to power: Quantifying the impact of generational mobile CPU design trends on performance, energy, and user satisfaction. Verfügbar 19. September 2025 unter <https://projects.iq.harvard.edu/files/mhalp16hpca.pdf>
- Jiang, Z., Chan, J., Tan, B. C., & Chua, W. S. (2010). Effects of Interactivity on Website Involvement and Purchase Intention. *Journal of the Association for Information Systems*, 11(1). <https://doi.org/10.17705/1jais.00218>
- Kellner, M. (2018, Dezember). *Leistung* [Enthält die selbst durchgeführten Leistungstests.]. Verfügbar 28. Oktober 2025 unter <https://github.com/MaximilianKellner/exploded-views-web-praxis-projekt/wiki/Leistung>
- Kokubun, H. (2024). *tweakpane: Compact GUI for fine-tuning parameters and monitoring value changes*. Verfügbar 11. Oktober 2025 unter <https://github.com/cocopon/tweakpane>
- McKeon, K. (2018, Dezember). *Top 6 Website Features People Value*. Clutch. Verfügbar 26. Oktober 2025 unter <https://clutch.co/resources/top-6-website-features-people-value>
- Nielsen, J. (1994a, April). *10 Usability Heuristics for User Interface Design*. Verfügbar 17. Oktober 2025 unter <https://www.nngroup.com/articles/ten-usability-heuristics/>
- Nielsen, J. (1994b, April). *Enhancing the explanatory power of usability heuristics*. <https://doi.org/10.1145/191666.191729>
- OnePlus. (2019). OnePlus 7 Pro [Explosionsansicht Kameramodul]. Verfügbar 24. August 2025 unter <https://www.oneplus.com/at/7pro#/>
- PwC. (2025, Juni). Verbreitung von Virtual-Reality-Hardware in Deutschland von 2019 bis 2024 und Prognose bis 2029 (in Millionen Stück). Verfügbar 21. August 2025 unter <https://de.statista.com/statistik/daten/studie/1247629/umfrage/virtual-reality-hardware-verbreitung-deutschland/>
- Rolon, J. N., Rangel, R. A., Curiel, R. F., Chia, G., & Kane, D. (n. d.). *Removable wheel assembly and method* (Patent application Nr. US 2025/0262893 A1). Verfügbar 21. August 2025 unter <https://ppubs.uspto.gov/api/pdf/downloadPdf/20250262893?requestToken=token>

- Teo, H.-H., Oh, L.-B., Liu, C., & Wei, K.-K. (2003). An empirical study of the effects of interactivity on web user attitude. *International Journal of Human-Computer Studies*, 58. [https://doi.org/10.1016/S1071-5819\(03\)00008-9](https://doi.org/10.1016/S1071-5819(03)00008-9)
- The LEGO Group. (2022). *Bauanleitung 10305: Burg der Löwenritter (1/2)*. Verfügbar 19. September 2025 unter <https://www.lego.com/cdn/product-assets/product.bi.core.pdf/6429213.pdf>
- The Visual Agency and The Veneranda Biblioteca Ambrosiana. (n. d.). *Codex Atlanticus* [Digitales Archiv des Codex Atlanticus von Leonardo da Vinci]. Verfügbar 24. August 2025 unter <https://www.codex-atlanticus.it/>